

答辩与评委提问应对指南 (Defense Q&A Guide)

本项目针对百万级数据（1,000,176 行原始文本，1,215,021 个语义分块）以及 AutoDL 算力卸载的实际落地，整理了答辩过程中各自板块评委可能会提及的高频疑难问题、答辩心法以及各自负责板块的“完美解答模板”，以保障汇报现场的高学术说服力。

一、 答辩总体核心策略与心法

- 突出数据科学工程规范：不要将项目描述成“简单调用大模型 API 的外壳”，要强调自研的 ETL (Ingest -> Clean -> Semantic Chunking)、本地 GPU 算力融合与 FastAPI 远程卸载、ChromaDB 原生 HNSW 检索、XSS 安全拦截网关以及 82 个自动化测试用例覆盖。这才是大数据与软件工程答辩的核心高分点。
- 逻辑清晰的“工程权衡 (Trade-Off)”叙事：评委常问“为什么不用更庞大的分布式方案/Milvus数据库？”。回答的核心是：在满足高校与企业特定规模知识检索的前提下，以零成本本地特征表征 + 极低廉的 API 开销换取 Recall@3 达 87.8% 的极高精度与安全防护保障，是一项高度成功的业务价值驱动的工程权衡。
- “故障即亮点”的排查素养展示：如果评委提到某些模块的局限性（如并发 429、ChromaDB 复合 where 过滤失效等），不要否认，而要大方展示我们在代码中设计的降级回退 (Fallback) 防护机制、自适应指数退避重试算法、以及在前台“玻璃盒”调试看板中透明打印中间状态的工程闭环，变危机为技术亮点的展示。

二、 各成员负责版块高频提问与解答

组长：李伟 — 系统架构与向量检索核心版块

Q1：你们的系统在稳态下端到端响应需要 6s，这在工业界能接受吗？瓶颈在哪？如何优化？

应对核心： 大模型网络延迟占 96%，本地检索仅占 4%。

回答模版： 本系统属于知识密集型多轮问答系统，包含查询意图解析和最终答案合成两重 LLM 推理。评委老师提到的 6s 耗时，在当前公有云 API 链路上是完全合理的。我们对“延迟预算 (Latency Budget)”进行了秒级定位，发现本地 GPU 向量化 (50ms) 与 ChromaDB 近邻检索 (200ms) 总耗时不足 4%，瓶颈完全在大模型公有云 API 调用上。优化路径包括：(1) 对高频意图建立 LRU 缓存；(2) 在线服务侧换用更轻量的本地大模型（如 Qwen1.5B-Instruct）；(3) 启用 Streamlit 前端流式字节输出 (Streaming Output)，消除用户等待焦虑。

Q2：你们的数据分块为什么不用固定长度切割？700 字符加 120 重叠是怎么得来的？

应对核心： 固定长度截断会撕裂语义，自研四层递进式语义分块提升召回率 28%。

回答模版： 固定字数切割会粗暴撕裂代码块、段落与 FAQ 的完整性。我们在自建金标测试集上实测表明，固定长度 (500 字符, 100 重叠) 的 Recall@3 仅为 62%，极易造成 FAQ 孤儿块丢失。为此，我们开发了四级退避语义分块算法（段落 -> 句子 -> 贪心拼接 -> 滑动切割）。自适应检测标点符号与物理段落边界，并辅以 120 字符重叠度。这完美保护了代码块物理边界，Recall@3 检索命中率大幅跃升至 87.8%。

Q3：当数据规模达到 100 万行时，ChromaDB 写入报错 DuplicateIDError 怎么解决的？

应对核心： 在 `embed_store.py` 中实现了组合 Chunk ID 的幂等 Upsert 机制。

回答模版： 在处理 100 万行级大规模数据建库时，由于文档源及切分块数量庞大，极易因文件名或分块索引重复触发 `DuplicateIDError`。我们在 `embed_store.py` 中重构了入库函数，废弃了简易的 `add` 方法，全面改写为 `upsert` 幂等写入，并引入了全局唯一的组合 Chunk ID 设计：`Chunk ID = f'{filename}_{doc_idx}_{idx}'`。这不仅消除了 ID 冲突，还实现了建库流程的幂等可重入性，允许我们在意外中断后一键断点续传。

数据工程师：张杰 — 多源语料采集与数据清洗工程版块**Q1：语料采集直接抓取 HTML 为什么不行？html2text 的作用是什么？**

应对核心： HTML 中的广告、侧边栏是严重噪音，转换为纯净 Markdown 能极大提升向量检索的信噪比。

回答模版： 互联网多源网页（如 CSDN 博客）包含大量的页眉页脚、广告推荐和脚本噪音。如果将原始 HTML 向量化，这些噪音将严重污染特征向量空间，降低召回率。我编写的采集引擎采用 `BeautifulSoup` 精准锁定核心内容 DOM 节点，并利用 `html2text` 转换为极为纯净的 Markdown 语法结构。这不仅消除了 90% 以上的无用噪音，还保留了标题层级和代码块边界，为后续的“语义感知分块”提供了高质量的数据基底。

Q2：你们 32 线程并发提取元数据时，遭遇公有云高频 API 限流（429 报错）怎么应对的？

应对核心： 指数退避自适应重试机制。

回答模版： 在高并发下大量文档同时发出元数据请求，极易触发 API 提供商的 HTTP 429 Rate Limit。我没有简单捕获报错导致任务中断，而是在代码内实现了一套带有随机抖动的自适应指数退避重试算法。当收到 429 信号时，线程会自动暂停并按 `2s -> 4s -> 8s` 的指数阶梯递增重试，最大重试 3 次。该策略大幅强化了预处理流水线在弱网与限流极端环境下的稳定运行能力，最终以 0.88 元的极低成本顺利完成了元数据批量提取。

Q3：AutoDL 远程显卡算力卸载是怎么配合本地流水线协同工作的？

应对核心： 本地 ETL 触发 -> 远程 FastAPI 接口并发推理 -> 本地 ChromaDB 写入。

回答模版： 百万级数据的向量特征生成在普通笔记本电脑上需要消耗数十小时。我们设计了“本地触发，云端卸载”的分布式计算方案。我编写了 `scripts/setup_autodl.sh` 脚本一键初始化云服务器，拉起由 FastAPI 驱动的高性能向量提取服务 `scripts/embedding_server.py`。本地 ETL 流水线清洗切块后，以 `batch_size=256` 批量发送 HTTP POST 请求，利用云端 RTX 4090 GPU 进行高并发推理，最终在 2.5 小时内以 4.70 元的租用成本，完成了全量特征计算，展现了极佳的云端协同工程表现。

前端开发工程师：王婷 — 前端现代美学交互与安全版块

Q1：大语言模型答复往往有十几秒的延迟，前端在 UI/UX 上做了哪些优化来提升用户体验？

应对核心：可折叠毛玻璃卡片、多轮对话状态机维护以及开发者“玻璃盒”剖析面板。

回答模版：为了避免用户在长达数秒的等待中产生系统崩溃的误判，我在前端交互设计上做出了三点优化：（1）在发起查询后立即拉起拟态的 Loading 动画，并在技术细节展示栏透明化呈现当前的意图解析中间状态（如提取出的 filters 字典）；（2）借用 session_state 建立平滑滚动聊天气泡，保持上下文关联；（3）设计了“开发者调试看板”，用户一键展开即可直观查看后台检索召回文本的余弦距离评分和原始文件名，将黑盒逻辑转化为透明的“玻璃盒”，极大提升了系统的可解释性与人机交互信任度。

Q2：知识库中如果被恶意注入指令，大模型的回答会受影响吗？前端做了哪些安全拦截？

应对核心：XSS 字符逃逸拦截与 HTML 实体转义。

回答模版：评委老师提到的指令注入与 XSS 攻击是 RAG 系统极易被忽视的安全漏洞。若外部语料中包含恶意的浏览器脚本或特殊转义指令，直接在前台渲染可能会引发 XSS 挂马劫持。为了保障系统沙箱安全，我在 Streamlit 变量输出网关上强引入了 Python 标准 html.escape() 进行 HTML 字符安全过滤，将恶意字符实体进行逃逸转义（如 < 转为 <）。同时，组长李伟在 System Prompt 中加入了强 Facts 限制，防止了指令注入穿透 LLM 的防线，构筑了前后端一体的安全过滤机制。

测试与文档工程师：刘洋 — 测试套件开发与技术文档保障版块

Q1：你们的测试套件有 82 个用例，但在没有网络和 API 密钥的测试环境下，如何保证测试顺利运行？

应对核心：全量 Mock 装饰器解耦，无物理网络依赖。

回答模版：在持续集成（CI）或离线测试环境下，网络超时或缺少私钥是阻碍自动化测试的最大痛点。为了解决这一问题，我全面采用了 Python 标准库中的 unittest.mock.patch 装饰器对外部 OpenAI/DeepSeek 接口、ChromaDB 连接以及 Hugging Face Embedding 下载环境实施了高保真度无物理依赖的 Mock 模拟。我为测试设计了固定的黄金返回数据和预期的异常抛出路，使得全套 82 个单测与集成测试用例可以在无网环境下一键在 3 秒内飞速跑完，并实现了 100% 成功通过，彻底证明了代码的高可靠性。

Q2：百万级建库需要 2.5 小时，如果本地数据库损坏了，你们怎么保证灾难恢复的？

应对核心：物理打包压缩、极速云盘同步（14MB/s）与一键幂等重建脚本。

回答模版：针对百万级数据重新建库耗时较长的问题，我们制定了双重容灾与备份恢复策略：（1）物理打包快照：由于 ChromaDB 采用 SQLite 和 Parquet 文件持久化，我主导设计了物理持久化文件夹 vector_store/ 的自动压缩容灾方案。打包为 5.8GB 的 .tar.gz 文件，通过 AliyunDrive 以 14MB/s 的极速上行链路同步，实现云端和本地的秒级镜像同步；（2）一键幂等重跑：如果数据库彻底损坏且快照丢失，我们在 src/main.py 中提供了一键重建命令，结合张杰开发的 32 线程预处理与 AutoDL 显卡并发卸载，可在 2.5 小时内幂等重构一个全新且数据一致的数据库。